

GENÔMICA COMPUTACIONAL

Projeto Prático

Calculadora de Ancestralidade Genômica com o 1000 Genomes

Análise de afinidade populacional com as 26 populações do 1000 Genomes

Objetivo central: compreender como dados de microarray são transformados em inferências populacionais, incluindo QC, harmonização, PCA, classificação, validação e interpretação crítica.

1. Visão geral do projeto

Neste trabalho, os grupos irão desenvolver uma calculadora computacional de ancestralidade e afinidade populacional a partir de dados reais de genotipagem humana. O sistema deverá receber o genótipo de um indivíduo obtido por meio de microarray da Illumina, compará-lo aos indivíduos de referência do 1000 Genomes Project e produzir uma análise de proximidade genética com as 26 populações do projeto.

- representação computacional de variantes genéticas;
- SNPs, genótipos e frequências alélicas;
- arquivos genômicos e genome builds;
- harmonização de variantes, alelos REF e ALT;
- controle de qualidade e dados ausentes;
- linkage disequilibrium e LD pruning;
- genética de populações e PCA;
- distâncias, classificação e otimização;
- visualização, validação e reprodutibilidade;
- interpretação biológica e limitações.

2. Problema

O ponto de partida é um arquivo com centenas de milhares de SNPs de um indivíduo genotipado por microarray da Illumina. Para cada SNP, o arquivo pode apresentar o rsID, o cromossomo, a posição e os dois alelos observados.

Index	Name	Address	Chr	Position	35.Genotipo
1	1:103380393	9663149	1	102914837	GG
2	1:109439680	26641362	1	108897058	AA
3	1:110198788	5682188	1	109656166	TT
4	1:110201112	21701591	1	109658490	CC

Observação: o arquivo fornecido nesta disciplina corresponde ao formato exportado do microarray Illumina, contendo as colunas Index, Name, Address, Chr, Position e 35.Genotipo. O campo 'Name' não deve ser tratado como rsID; a integração com o 1000 Genomes deverá ser realizada principalmente pelas coordenadas genômicas (Chr e Position), obtendo posteriormente REF e ALT para a harmonização.

3. As 26 populações do 1000 Genomes

AFR — African

Código	População
ACB	African Caribbeans in Barbados
ASW	Americans of African Ancestry in SW USA
ESN	Esan in Nigeria
GWD	Gambian in Western Divisions in The Gambia
LWK	Luhya in Webuye, Kenya
MSL	Mende in Sierra Leone
YRI	Yoruba in Ibadan, Nigeria

AMR — Admixed American

Código	População
CLM	Colombians from Medellín, Colombia
MXL	Mexican Ancestry from Los Angeles USA
PEL	Peruvians from Lima, Peru
PUR	Puerto Ricans from Puerto Rico

EAS — East Asian

Código	População
CDX	Chinese Dai in Xishuangbanna, China
CHB	Han Chinese in Beijing, China
CHS	Southern Han Chinese
JPT	Japanese in Tokyo, Japan
KHV	Kinh in Ho Chi Minh City, Vietnam

EUR — European

Código	População
CEU	Utah Residents with Northern and Western European Ancestry
FIN	Finnish in Finland
GBR	British in England and Scotland
IBS	Iberian Population in Spain
TSI	Toscani in Italy

SAS — South Asian

Código	População
BEB	Bengali from Bangladesh

GIH	Gujarati Indian from Houston, Texas
ITU	Indian Telugu from the UK
PJL	Punjabi from Lahore, Pakistan
STU	Sri Lankan Tamil from the UK

O conjunto Phase 3 clássico do 1000 Genomes contém 2.504 indivíduos pertencentes a essas 26 populações.

4. Interpretação científica

Uma população do 1000 Genomes não representa necessariamente uma ancestralidade ancestral pura. Populações como PUR, CLM, MXL e PEL são populações contemporâneas com histórias demográficas próprias e diferentes graus de miscigenação.

Portanto, um resultado como:

```
IBS  35%
TSI  24%
PUR  15%
YRI  8%
...
```

não deve ser interpretado automaticamente como uma porcentagem literal de nacionalidade. Dependendo do método, os valores representam afinidade genética relativa ou uma composição simplificada em relação às populações de referência disponíveis.

5. Preparação prévia da referência

Para esta edição da disciplina, a preparação da referência foi realizada previamente pelo professor a partir dos dados do 1000 Genomes Project (GRCh38) e do manifesto oficial do Illumina Global Screening Array v3 (GSA v3). Essa etapa demandou elevado processamento computacional e, portanto, não fará parte das atividades dos alunos.

- Filtragem do 1000 Genomes para os marcadores do Illumina GSA v3.
- Seleção apenas de SNPs bialélicos.
- Harmonização entre os alelos do microarray e os alelos REF/ALT da referência.
- Correção de diferenças de orientação (strand).
- Remoção de variantes incompatíveis.
- Geração da referência reduzida utilizada na disciplina.

Como resultado, obteve-se uma referência contendo 573.028 SNPs harmonizados, com correspondência única entre os marcadores do microarray e as variantes do 1000 Genomes.

6. Organização dos dados no Google Drive

https://drive.google.com/drive/folders/10_gf_A5gl-cZ99mRlOflo4wmrl77ojPP?usp=drive_link

```
├─ individuo/
│   └─ genotipo_microarray.csv
├─ manifesto/
│   └─ GSA-24v3-0-A2-manifest-file-csv.zip
├─ referencia/
│   ├── all_hg38.pgen.zst
│   ├── all_hg38_rs.pvar.zst
│   └─ hg38_corrected.psam
└─ processed/
    ├── 1000G_GSA_SNP.pgen
    ├── 1000G_GSA_SNP.pvar.zst
    └─ 1000G_GSA_SNP.psam
```

7. Conectando o Colab ao Google Drive

Antes de executar o notebook, abra a pasta no Google Drive e use **Adicionar atalho ao Drive**, escolhendo o **Meu Drive** como destino. Assim, o Colab poderá acessar a pasta pelo mesmo caminho em todas as contas.

```
from google.colab import drive
from pathlib import Path

drive.mount('/content/drive')

BASE = Path('/content/drive/MyDrive/Projeto_Ancestralidade')
INDIVIDUO = BASE / 'individuo'
REFERENCIA = BASE / 'referencia'      # arquivos originais (rastreadabilidade)
MANIFESTO = BASE / 'manifesto'
PROCESSED = BASE / 'processed'        # arquivos utilizados no projeto

print('BASE =', BASE)
GENOTIPO = INDIVIDUO / 'genotipo_microarray.csv'

PGEN = PROCESSED / '1000G_GSA_SNP.pgen'
PVAR = PROCESSED / '1000G_GSA_SNP.pvar.zst'
PSAM = PROCESSED / '1000G_GSA_SNP.psam'

arquivos = [
    GENOTIPO,
    PGEN,
    PVAR,
    PSAM,
]

print('\nVerificando arquivos necessários:\n')
```

```
for arquivo in arquivos:
    status = '✓' if arquivo.exists() else 'X'
    print(f'{status} {arquivo}')
```

8. Matriz genotípica

Precisamos de uma matriz genotípica $X \in \mathbb{R}^{(N \times M)}$, onde N representa o número de indivíduos da referência e M o número de SNPs utilizados nas análises.

Indivíduo	SNP1	SNP2	SNP3	SNP4
HG00096	0	1	2	0
HG00097	1	1	2	0
HG00100	0	0	1	0
INDIVÍDUO	1	0	2	0

8.1 Leitura da referência preparada

A referência preparada pelo professor está armazenada no formato PLINK 2 e é composta pelos arquivos localizados na pasta processed:

- processed/1000G_GSA_SNP.pgen
- processed/1000G_GSA_SNP.pvar.zst
- processed/1000G_GSA_SNP.psam

O arquivo .pgen contém os genótipos, o .pvar.zst descreve as variantes e o .psam contém os metadados dos indivíduos da referência.

Informação	Quantidade
Indivíduos	3.202
SNPs bialélicos da referência	575.480

Durante os testes da disciplina foi obtido:

Dimensão da matriz: (3202, 575480)

Tipo: int8

Os genótipos são codificados em relação ao alelo ALT:

Código	Significado
0	REF/REF
1	REF/ALT
2	ALT/ALT
-9	Genótipo ausente

8.2 Mapa de harmonização

Nem todos os SNPs presentes na referência são utilizados diretamente nas análises. Foi construída previamente uma correspondência única entre os marcadores do Illumina GSA v3 e as variantes do 1000 Genomes.

processed/harmonization_map.csv

Após a remoção de variantes incompatíveis e de marcadores duplicados, o conjunto final utilizado na disciplina contém 572.011 SNPs únicos.

Campo	Descrição
INDEX	Índice da variante em X_ref
Name	Marcador Illumina
REF	Alelo de referência
ALT	Alelo alternativo
NEEDS_COMPLEMENT	Indica necessidade de complementar os alelos

A matriz utilizada nas análises passa a ser:

```
X_ref_final.shape = (3202, 572011)
```

9. Preparação do Genoma Testado

O arquivo 'genotipo_microarray.csv' contém os genótipos do indivíduo. Esses dados devem ser convertidos para a mesma representação utilizada na referência.

A conversão utiliza exclusivamente o arquivo 'processed/harmonization_map.csv', não sendo necessário repetir a harmonização entre o microarray e o 1000 Genomes.

genotipo_microarray.csv



harmonization_map.csv



orientação dos alelos



codificação 0 / 1 / 2



X_individuo

O vetor final deverá possuir:

```
X_individuo.shape = (572011,)
```

Código	Significado
0	REF/REF
1	REF/ALT
2	ALT/ALT
-9	Ausente ou incompatível

Ao final desta etapa, a referência e o indivíduo estarão representados no mesmo espaço genotípico:

```
X_ref_final.shape = (3202, 572011)
```

```
X_individuo.shape = (572011,)
```

10. Frequências alélicas

Após a construção da matriz `X_ref_final`, calcula-se a frequência do alelo ALT para cada SNP da referência. Essas frequências serão utilizadas posteriormente na imputação de dados ausentes e em análises exploratórias.

A frequência é calculada por:

$p = \text{soma}(\text{dosagens ALT}) / (2 \times \text{número de indivíduos válidos})$
Ignorando genótipos ausentes (-9), obtém-se um vetor:

`freq_alt.shape = (572011,)`

Para um SNP, $p = \text{número de alelos ALT} / (2N)$. Com genótipos codificados como 0, 1 e 2:

11. Frequências para as 26 populações

```
frequencias = {}

for pop in metadata["population"].unique():
    individuos = metadata.loc[
        metadata["population"] == pop,
        "sample"
    ]

    X_pop = X.loc[X.index.intersection(individuos)]
    frequencias[pop] = X_pop.mean(axis=0) / 2

freq_df = pd.DataFrame(frequencias)
```

12. SNPs informativos para ancestralidade

Uma medida didática simples de diferenciação populacional é $\Delta p_i = \max_k(p_{ik}) - \min_k(p_{ik})$.

```
delta_p = freq_df.max(axis=1) - freq_df.min(axis=1)
delta_p.sort_values(ascending=False).head(20)
```

Comparem análises com todos os SNPs e subconjuntos de 50.000, 20.000, 10.000, 5.000 e 1.000 SNPs.

13. Linkage Disequilibrium e LD pruning

SNPs próximos podem estar fortemente correlacionados. Uma análise de PCA ou estrutura populacional pode se beneficiar de LD pruning.

```
plink \
  --bfile referencia \
  --indep-pairwise 50 5 0.2 \
  --out pruning

plink \
  --bfile referencia \
  --extract pruning.prune.in \
  --make-bed \
  --out referencia_pruned
```

14. Parte I — Análise exploratória por PCA

A PCA será a primeira análise populacional obrigatória.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=10, random_state=42)
coords = pca.fit_transform(X_ref)

for i, v in enumerate(pca.explained_variance_ratio_, start=1):
    print(f"PC{i}: {v*100:.2f}%")
```

15. Gráficos de PCA

```
pca_df = pd.DataFrame(
    coords[:, :2],
    columns=["PC1", "PC2"],
    index=X_ref.index
).join(metadata)

import matplotlib.pyplot as plt

for pop in pca_df["population"].unique():
    subset = pca_df[pca_df["population"] == pop]
    plt.scatter(
        subset["PC1"], subset["PC2"],
        label=pop, s=12, alpha=0.7
    )

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend(bbox_to_anchor=(1.05, 1), loc="upper left")
plt.show()
```

Produzam versões por superpopulação e por 26 populações.

16. Projetando o indivíduo na PCA

A PCA deverá ser ajustada somente com a referência; depois, projete o indivíduo.

```
pca.fit(X_ref)

ref_coords = pca.transform(X_ref)
ind_coords = pca.transform(
    X_individuo.reshape(1, -1)
)
```

1. Em qual região da PCA o indivíduo aparece?
2. Qual superpopulação parece mais próxima?
3. Quais populações específicas aparecem mais próximas?
4. PC1 × PC2 é suficiente?
5. O que muda em PC2 × PC3 e PC3 × PC4?

17. Parte II — Distância aos centroides populacionais

```
pc_cols = [
    "PC1", "PC2", "PC3", "PC4", "PC5",
    "PC6", "PC7", "PC8", "PC9", "PC10"
]

centroides = (
    pca_df
    .groupby("population")[pc_cols]
    .mean()
)

import numpy as np

ind = ind_coords[0]
distancias = {}

for pop, centro in centroides.iterrows():
    distancias[pop] = np.linalg.norm(
        ind - centro.values
    )

ranking = pd.Series(distancias).sort_values()
ranking.head(10)
```

Esse ranking mede proximidade no espaço PCA. Ele não representa porcentagens biológicas de ancestralidade.

18. Score de afinidade relativa

Opcionalmente, as distâncias podem ser transformadas em um score relativo, por exemplo $s_k = \exp(-d_k/T)$.

```
T = 1.0

d = ranking.values
scores = np.exp(-d / T)
scores = scores / scores.sum()

similaridade = pd.Series(scores, index=ranking.index)
similaridade.sort_values(ascending=False)
```

19. Parte III — Classificação supervisionada

Implementem pelo menos um classificador: k-NN, regressão logística ou Random Forest.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_ref,
    y_pop,
    test_size=0.20,
    stratify=y_pop,
    random_state=42
)
```

```

from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=15,
    weights="distance"
)

knn.fit(X_train_pca, y_train)
pred = knn.predict(X_test_pca)

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

print(accuracy_score(y_test, pred))
print(classification_report(y_test, pred))

```

20. Classificação hierárquica

Comparem uma classificação direta das 26 populações com uma estratégia em dois níveis: primeiro AFR/AMR/EAS/EUR/SAS e depois a população dentro da superpopulação selecionada.

21. Probabilidades do classificador

Alguns modelos fornecem `predict_proba()`. Probabilidade de classe não equivale a porcentagem de ancestralidade genética.

22. Parte IV — Modelo simplificado de mistura por frequências

Para cada SNP i , seja p_{ik} a frequência do alelo ALT na população k . Buscamos pesos $w_1, \dots, w_{26}$ com $w_k \geq 0$ e soma igual a 1.

A frequência esperada é $\hat{p}_i = \sum_k w_k p_{ik}$ e o genótipo esperado é $\hat{g}_i = 2\hat{p}_i$. Minimiza-se a diferença quadrática entre g_i e $\hat{g}_i$.

```

import numpy as np
from scipy.optimize import minimize

K = P.shape[1]

def objective(w):
    esperado = 2 * P.dot(w)
    return np.mean((g - esperado) ** 2)

w0 = np.ones(K) / K
bounds = [(0, 1) for _ in range(K)]

constraints = {
    "type": "eq",
    "fun": lambda w: np.sum(w) - 1
}

resultado = minimize(
    objective,
    w0,
    method="SLSQP",
    bounds=bounds,
    constraints=constraints
)

print(resultado.success)

pesos = pd.Series(
    resultado.x,
    index=populacoes
).sort_values(ascending=False)

pesos

```

Esse modelo é uma simplificação didática e não substitui métodos especializados de inferência de admixture.

23. Limitações do modelo de mistura

- correlação entre SNPs;
- populações de referência geneticamente semelhantes;
- frequências estimadas em amostras finitas;
- linkage disequilibrium;
- estrutura populacional e deriva genética;
- desequilíbrio amostral entre populações;
- populações contemporâneas não representam ancestrais puros;
- soluções potencialmente não únicas;
- sensibilidade à seleção de SNPs.

24. Resultado agregado por superpopulação

```
pop_to_super = {
    "ACB": "AFR", "ASW": "AFR", "ESN": "AFR", "GWD": "AFR", "LWK": "AFR", "MSL": "AFR", "YRI": "AFR",
    "CLM": "AMR", "MXL": "AMR", "PEL": "AMR", "PUR": "AMR",
    "CDX": "EAS", "CHB": "EAS", "CHS": "EAS", "JPT": "EAS", "KHV": "EAS",
    "CEU": "EUR", "FIN": "EUR", "GBR": "EUR", "IBS": "EUR", "TSI": "EUR",
    "BEB": "SAS", "GIH": "SAS", "ITU": "SAS", "PJI": "SAS", "STU": "SAS"
}

resultado_pop = pesos.to_frame("score")
resultado_pop["superpop"] = resultado_pop.index.map(pop_to_super)

resultado_super = (
    resultado_pop
    .groupby("superpop")["score"]
    .sum()
    .sort_values(ascending=False)
)

resultado_super
```

25. Saída final da calculadora

```
=====
ANÁLISE GENÔMICA POPULACIONAL
=====

SNPs no arquivo:                654.027
SNPs encontrados no 1000G:      612.381
SNPs após harmonização:        598.472
SNPs após QC:                   573.821
SNPs após LD pruning:          72.314

SUPERPOPULAÇÕES
EUR      72.8%
AMR      17.1%
AFR       7.0%
SAS       2.1%
EAS       1.0%

POPULAÇÕES DE REFERÊNCIA
IBS      31.4%
TSI      20.8%
GBR      11.2%
PUR       8.1%
CEU       7.4%
CLM       6.3%
YRI       4.2%
...
```

Os números acima são apenas ilustrativos.

26. Visualizações obrigatórias

1. gráfico de barras das 26 populações;

2. gráfico agregado das cinco superpopulações;
3. PCA PC1 × PC2 por superpopulação com o indivíduo destacado;
4. PCA das 26 populações;
5. PCA focal nas cinco populações mais próximas.

```

resultado = pesos * 100

resultado.plot(kind="bar", figsize=(15, 5))
plt.ylabel("Contribuição relativa (%)")
plt.xlabel("População 1000 Genomes")
plt.tight_layout()
plt.show()

```

27. Validação interna

1. escolha um indivíduo do 1000 Genomes;
2. remova esse indivíduo da referência;
3. trate-o como desconhecido;
4. execute a calculadora;
5. compare população real e população prevista;
6. repita para vários indivíduos.

Calcule Top-1 accuracy, Top-3 accuracy e accuracy de superpopulação. Construa também uma matriz de confusão 26 × 26.

28. Comparação entre métodos

Método	Resultado principal	Vantagens	Limitações
PCA + centroides			
Classificador			
Mistura por frequências			

29. Pergunta final do projeto

Até que ponto é possível inferir a ancestralidade genética de um indivíduo utilizando um microarray comercial e as 26 populações do 1000 Genomes como conjunto de referência?

A resposta deverá considerar composição da referência, resolução populacional, número e seleção dos SNPs, LD, incerteza estatística, miscigenação e limitações das 26 populações.

30. Fluxo conceitual esperado

